

The Crossover Payload of an Encrypted UDP Data Plane

Qiyue Zhao

Abstract. An encrypted tunnel endpoint pays two kinds of cost for every packet it forwards: a fixed cost, which does not depend on the payload, and a marginal cost, which grows with every payload byte. Which of the two dominates determines whether batching or a faster cipher will accelerate the data plane, yet throughput measurements conflate the two costs and cipher benchmarks omit the packet path altogether. In this paper, we decompose the per-packet cost of an encrypted UDP data plane into a fixed term a and a per-byte term b , and characterise the data plane by their ratio, the crossover payload $s^* = a/b$, which is invariant to the unit of both terms. Specifically, we fit this model over 104 configurations that cover four AEAD implementations, hardware cryptographic instructions present and masked, and payloads up to the largest that a 1,500-byte MTU admits, with confidence intervals propagated through the covariance of the fit. Moreover, we apply the same decomposition at the system-call boundary, at the isolation layer around a virtual network function and at the bus to a GPU. For our AES-NI implementation of AES-256-GCM, $a = 6,768$ cycles and $b = 1.869$ cycles per byte, so $s^* = 3,621$ bytes lies 2.5 times beyond the largest carriable payload of 1,432 bytes. The per-byte term thus accounts for at most 28.3% of the cost of any packet, which bounds the benefit of any faster cipher. A null-cipher control further attributes 1,642 of the 7,982 cycles of fixed cost with OpenSSL’s AES-256-GCM to the AEAD itself.

Keywords: Data plane · Authenticated encryption · Cost model · Performance measurement · Kernel bypass.

1 Introduction

A tunnel endpoint that terminates one security association and originates another pays two kinds of cost for every packet it forwards. The first kind does not depend on the size of the packet. It comprises the system calls that move the packet across the kernel boundary, the parsing of its header, the anti-replay check of ESP [8] and its successors, the derivation of the nonce, and the per-message setup and teardown of the cipher. The second kind grows with the payload. It comprises the bulk encryption and decryption, together with the memory traffic that feeds them.

The two kinds of cost call for opposite optimisations. The fixed cost is reduced by batching and by removing system calls, as kernel-bypass frameworks and shared submission rings do [2,3,13]. The per-byte cost is reduced by a faster

cipher or by fewer passes over the payload [6,9]. Which of these efforts pays off therefore depends on which cost dominates, but the measurements that are usually reported do not reveal it. A throughput figure at a single packet size conflates the two costs, and a cipher benchmark such as `openssl speed` times the cipher without the packet path around it.

Therefore, we separate the two costs explicitly. We model the cost of forwarding one packet with a payload of s bytes as

$$C(s) = a + b s, \quad (1)$$

where a is the fixed cost in cycles and b is the cost in cycles per payload byte, and we characterise the data plane by the crossover payload $s^* = a/b$, at which the two terms are equal. Below s^* , a packet costs mostly what it costs to be a packet; above s^* , it costs mostly what it costs to carry its bytes. The crossover payload is also more portable than either of its terms. Since a and b are expressed in the same unit, any rescaling of that unit, for example by a different effective clock rate, cancels in their ratio, whereas the absolute values of a and b belong to the machine that produced them.

We measure a decrypt-then-re-encrypt forwarding element, in which every payload byte crosses the cipher twice, which makes b large enough to be separated cleanly from a . We vary four factors: the AEAD implementation, the availability of hardware cryptographic instructions, the transport and the payload size. For AES-256-GCM [5] and ChaCha20-Poly1305 [12], we measure both the OpenSSL implementation and an implementation written for this study. The second implementation separates the cost of an algorithm from the cost of one library’s version of it, and masking the hardware instructions in both implementations isolates the contribution of the accelerator.

Contribution. This paper characterises the per-packet cost of an encrypted UDP data plane by its crossover payload. We fit the cost model by weighted least squares over 104 configurations and propagate the uncertainty of s^* through the covariance of a and b . We show that the per-byte term accounts for the fraction $s/(s+s^*)$ of the cost at payload s , so that a crossover beyond the largest payload bounds the benefit of any per-byte optimisation. For our AES-NI implementation of AES-256-GCM, the crossover is 3,621 bytes and this bound is 28.3%.

We apply the same fixed-versus-marginal decomposition at three further boundaries. At the system-call boundary, we measure four transports, including an `io_uring` path that submits packets without any system call. At the isolation boundary, we measure the cost of a network namespace and of a chain of encrypting network functions. At the bus, we derive the batch size below which a GPU offload cannot outperform the processor, however fast its kernel. Finally, we validate the measurement with five controls whose predictions are fixed before they run, including a null cipher that attributes 1,642 cycles of the fixed cost to the AEAD itself.

The rest of this paper is organised as follows. Section 2 reviews related work, Section 3 presents the cost model and its extension to further boundaries, and

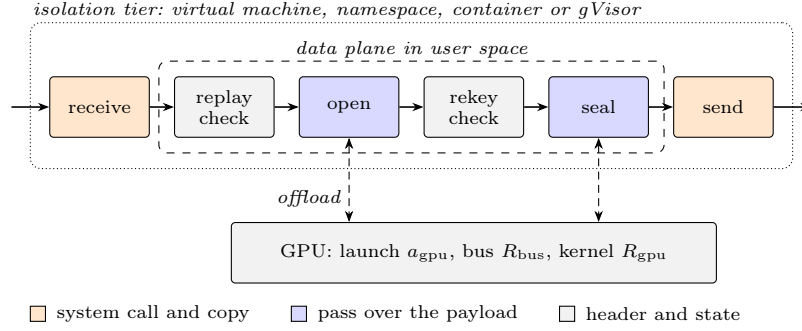


Fig. 1. Overview of the measured forwarding path and of the boundaries studied in this paper. Each packet crosses the system-call boundary on receive and on send, and each payload byte crosses the cipher twice. The fixed term a collects the system calls, the header parsing, the anti-replay window, the nonce derivation and the per-message setup of the AEAD; the per-byte term b collects the two cipher passes and the memory traffic around them. The same decomposition is applied at the isolation tier around the data plane and at the bus to a GPU.

Section 4 reports the experimental evaluation. Section 5 discusses the limitations of this study, and Section 6 concludes.

2 Related Work

Batched and Kernel-Bypass Packet I/O. Netmap [13] maps the packet buffers of a network interface into user space and amortises each system call over a batch of packets. Barbette et al. [3] compare user-space packet I/O frameworks quantitatively and derive design principles for software packet processors on multi-core hardware. `io_uring` [2] shares submission and completion rings between a process and the kernel, and with a kernel-side poller, a process can submit I/O without issuing a system call. These systems reduce the fixed cost of a packet, but they do not relate it to the per-byte cost that encryption adds to the data path.

Authenticated Encryption on the Data Path. ESP [8] protects IP packets with a sequence number and a sliding anti-replay window, and WireGuard [4] implements an encrypted tunnel inside the kernel. The AEADs that such tunnels use, AES-GCM [5] and ChaCha20-Poly1305 [12], owe their speed to dedicated instructions, such as carry-less multiplication for GHASH [6], and to vectorised implementations [9]. Cipher benchmarks report the bulk rate of these implementations, which characterises the per-byte term but reveals nothing about the fixed cost of the packet that surrounds the cipher.

Isolation and Virtualised Network Functions. Manousis et al. [10] show that co-resident network functions contend for several distinct components of

the memory subsystem, and they predict the resulting performance degradation. Young et al. [15] quantify the cost of gVisor, which services the system calls of a container in a user-space kernel. Both lines of work concern the cost of sharing or isolating a machine. We ask a narrower question, namely whether an isolation layer changes the fixed term of a packet, its per-byte term, or both.

Performance Models and Benchmarking. The Roofline model [14] bounds attainable performance by the compute capacity and the memory bandwidth of a machine, and the STREAM benchmark [11] measures that bandwidth. Hoefer and Belli [7] recommend summarising performance measurements by medians with nonparametric confidence intervals, because such measurements are rarely normally distributed. We follow this recommendation, and we derive a bound in the spirit of the Roofline model for the rate at which a data plane can move payload.

3 The Cost Model

This section defines the measured path and the cost model (Section 3.1), describes how the model is estimated (Section 3.2), and applies the same decomposition to further boundaries (Section 3.3).

3.1 Measured Path and Cost Decomposition

Every experiment times the same forwarding loop, shown in Fig. 1: receive, replay check, open, rekey check, seal and send. A difference between two measurements is therefore attributable to the factor that was varied rather than to a different amount of work. The replay check precedes decryption, so that a replayed packet cannot cost a decryption, and the cost of the replay window consequently belongs to the fixed term.

A packet carries a 24-byte header and a 16-byte authentication tag. The nonce is derived from the epoch and the sequence number rather than carried in the packet, as deployed protocols do, which keeps the header at 24 bytes. The header is authenticated as associated data, so a modified sequence number, epoch or timestamp is rejected together with a modified payload. With 28 bytes of IPv4 and UDP headers, the largest payload that fits a 1,500-byte MTU is

$$s_{\max} = 1,500 - 28 - 24 - 16 = 1,432 \text{ bytes.} \quad (2)$$

The cost model (1) assigns every piece of work on this path to one of its two terms. The fixed term a collects the system calls, the header parsing, the replay window, the nonce derivation and the per-message setup of the AEAD. The per-byte term b collects the two passes of the payload through the cipher and the memory traffic around them. We summarise the model by the following quantity.

Definition 1 (Crossover payload). *The crossover payload of a data plane with cost model (1) is $s^* = a/b$, the payload at which its fixed and per-byte terms are equal.*

Proposition 1 (Per-byte share). *For every payload $s \geq 0$, the per-byte term accounts for the fraction*

$$\rho(s) = \frac{bs}{C(s)} = \frac{s}{s + s^*} \quad (3)$$

of the per-packet cost. Consequently, if $s^ > s_{\max}$, the fixed term exceeds the per-byte term at every carriable payload, and eliminating the per-byte term entirely reduces the cost of any carriable packet by at most $s_{\max}/(s_{\max} + s^*)$.*

Proof. Dividing the numerator and the denominator of $bs/(a + bs)$ by b gives $s/(s + s^*)$. This fraction is below $1/2$ exactly when $s < s^*$, and it increases with s , so on $[0, s_{\max}]$ it attains its maximum at $s_{\max}$.

Proposition 1 turns the crossover payload into a bound on the benefit of any per-byte optimisation, such as a faster cipher or fewer copies of the payload. The bound depends on a and b only through s^* , so it is unaffected by the unit in which cost is measured.

3.2 Estimating the Model

We measure every configuration in 11 replicates, each of which reports the cycles per packet over a window of many packets. The replicate count is odd, so that the median is an observed value rather than the mean of two. We summarise each configuration by its median and by a percentile bootstrap interval on the median rather than by a standard deviation, because the per-packet cost is right-skewed by scheduling events and a symmetric interval would misrepresent it [7]. A configuration whose coefficient of variation exceeds 10% is marked unstable and excluded from the fit.

Weighted Fit. We estimate a and b by weighted least squares over the stable configurations, with weight $w_i = 1/h_i^2$ for a configuration whose bootstrap interval has half-width h_i . The weights matter because small payloads yield the noisiest configurations: scheduling noise lands in the fixed cost, and an unweighted fit would let these configurations determine the intercept a , on which the crossover payload depends.

Interval on the Crossover Payload. Since $\hat{a}$ and $\hat{b}$ are estimated from the same data, they are strongly anti-correlated, because a fit that raises the intercept lowers the slope. We therefore propagate the uncertainty of $\hat{s}^* = \hat{a}/\hat{b}$

through the covariance of the fit rather than dividing the separate intervals of $\hat{a}$ and $\hat{b}$, which would ignore this dependence. To first order,

$$\text{Var}(\hat{s}^*) \approx (s^*)^2 \left(\frac{\text{Var}(\hat{a})}{a^2} + \frac{\text{Var}(\hat{b})}{b^2} - \frac{2 \text{Cov}(\hat{a}, \hat{b})}{a b} \right), \quad (4)$$

and we report the 95% interval $\hat{s}^* \pm t_{0.975, m-2} \text{Var}(\hat{s}^*)^{1/2}$, where m is the number of stable configurations.

Curvature Test. A linear fit returns some a and b for any data. To detect a departure from linearity, we also fit a model with a quadratic term, compare the two nested models with an F -test, and report the quadratic term whenever it is significant at the 1% level.

3.3 The Same Decomposition at Further Boundaries

System-Call Boundary. We run the same forwarding loop over four transports whose system-call rates differ by construction. `posix` issues one receive call and one send call per packet, and `mmmsg` issues one `recvmmmsg` and one `sendmmmsg` call per batch of B packets. `uring` batches submissions through a ring shared with the kernel [2], and `uring-sqpoll` adds a kernel-side submission poller, so that submitting a batch requires no system call at all. Without such a path, the contribution of the system-call boundary to a can only be extrapolated from the shape of a batching curve. With it, and provided that the poller does not compete with the data plane for a core, the difference between `posix` and `uring-sqpoll` at $B = 1$ measures this contribution directly.

Isolation Boundary. A virtual network function runs inside an isolation tier. We consider four tiers: the kernel of the virtual machine with its paravirtualised network interface, a network namespace joined by a veth pair, a runc container behind a bridge with network address translation, and a gVisor sandbox, in which a user-space kernel services the system calls of the function [15]. Each tier is expected to add fixed cost per packet without adding per-byte cost, in the same way as a system call one level further out, and the gVisor tier is expected to move a the most, because every system call on the path then crosses into another process. A service function chain composes such functions: each hop terminates one security association and originates another, and the slope of the cost against the length of the chain gives the price of composition.

Bus. Offloading the AEAD to a GPU has the shape of (1) with different constants. Processing a batch of k packets with payload s on the GPU takes

$$T_{\text{gpu}}(k, s) = a_{\text{gpu}} + \frac{k s}{R_{\text{bus}}} + \frac{k s}{R_{\text{gpu}}}, \quad (5)$$

where a_{gpu} is the time to launch the kernel and to synchronise with it, R_{bus} is the bandwidth of the bus, and R_{gpu} is the rate of the kernel. The middle term is a transfer cost that the processor does not pay at all. Assuming that per-message costs are equal on both sides, the processor performs the same work in $T_{\text{cpu}}(k, s) = k s / R_{\text{cpu}}$, where R_{cpu} is its bulk rate.

Proposition 2 (Minimum batch for offload). *Offloading a batch of k packets with payload s is faster than processing it on the processor if and only if*

$$\frac{1}{R_{\text{bus}}} + \frac{1}{R_{\text{gpu}}} < \frac{1}{R_{\text{cpu}}} \quad \text{and} \quad k > \frac{a_{\text{gpu}}}{s(1/R_{\text{cpu}} - 1/R_{\text{bus}} - 1/R_{\text{gpu}})}. \quad (6)$$

In particular, no kernel rate R_{gpu} makes the offload faster unless $R_{\text{bus}} > R_{\text{cpu}}$, and none makes it faster for a batch of at most

$$k_{\min}(s) = \frac{a_{\text{gpu}} R_{\text{cpu}} R_{\text{bus}}}{s(R_{\text{bus}} - R_{\text{cpu}})} \quad (7)$$

packets.

Proof. By (5), $T_{\text{gpu}} < T_{\text{cpu}}$ is equivalent to $k s (1/R_{\text{cpu}} - 1/R_{\text{bus}} - 1/R_{\text{gpu}}) > a_{\text{gpu}}$. Since $a_{\text{gpu}} > 0$, the bracket must be positive, and k must then exceed a_{gpu} divided by s times the bracket, which gives (6). As R_{gpu} grows, the bracket increases towards $1/R_{\text{cpu}} - 1/R_{\text{bus}}$, which is positive only if $R_{\text{bus}} > R_{\text{cpu}}$, and the threshold on k decreases towards $k_{\min}(s)$ without reaching it.

Proposition 2 is the counterpart of Proposition 1 at the bus. The launch time a_{gpu} is a fixed cost that only a sufficiently large batch amortises, and the transfer acts as a per-byte cost that the processor does not pay, so a faster kernel cannot compensate for a bus that is slower than the processor.

4 Experimental Evaluation

4.1 Experimental Setup

Implementation and Platform. We implemented the data plane, which we call `linerate`, in C++. The data plane takes `aes-256-gcm` and `chacha20-poly1305` from OpenSSL 3.0.13 and contains the two implementations written for this study, `1r-aes-256-gcm` and `1r-chacha20-poly1305`. The first of these combines AES-NI, a GHASH based on carry-less multiplication [6], and a single-pass loop that interleaves counter-mode encryption with the hash. The second vectorises ChaCha20 with AVX2. Both produce output identical to that of OpenSSL at every payload length in the sweep. All measurements run on a virtual machine with an Intel Xeon Platinum 8481C processor at 2.70 GHz, two physical cores with simultaneous multithreading and a gVNIC network interface, under Linux 6.17. The code is compiled by GCC 13.3.0 with `-O3 -march=native`. The network references of Section 4.5 run between two such machines.

Measurement Loop. Within each measurement unit, a loader stages a window of encrypted packets in the receive buffer of the data-plane socket, and the data plane then drains the window through the forwarding loop of Fig. 1. Only the drain is timed, so generating packets never contributes to the measured cost, and warm-up windows, which populate the caches, the branch predictors and the page tables, are discarded. Cost is expressed in reference cycles of the invariant time-stamp counter, which advances at the nominal clock rate of 2.700 GHz, and each unit divides the cycles of its timed region by the number of packets that it forwarded.

Sweep Design. Each configuration is measured in 11 replicates (Section 3.2). All replicates of all configurations are interleaved in a single randomised order rather than executed in nested loops. In a nested loop, a machine that warms up or acquires a noisy neighbour halfway through would bias whichever factor level was running at the time, whereas an interleaved order spreads such drift across all levels. An anchor configuration is measured at the start and at the end of every sweep, each time as the median of five units, and a sweep whose two anchors differ by more than 10% is discarded; in the sweep reported here, the anchor moved by 0.26%. The hardware cryptographic instructions are masked by clearing their capability bits, which both OpenSSL and our dispatcher read when they are loaded. Each unit therefore runs in a fresh process with the mask set or cleared, and this process confirms that the mask has taken effect before it measures.

4.2 Cost Model and Crossover Payload

Figure 2 shows the cost of all 104 configurations, and Table 1 lists the fitted models. Here and in the remainder of this paper, implementation names without a prefix denote OpenSSL, and the prefix `1r-` denotes the implementations written for this study. The linear model describes every cell but one: for `1r-aes-256-gcm` with the instructions masked, the quadratic term is significant at the 1% level, so the fitted a and b of this cell describe the measured payload range rather than the implementation as a whole.

With the hardware instructions available, `1r-aes-256-gcm` has $a = 6,768$ cycles and $b = 1.869$ cycles per byte, which gives $s^* = 3,621$ bytes with a 95% interval of $[3,505, 3,736]$ (Fig. 3). The crossover therefore lies 2.5 times beyond $s_{\max}$, and by Proposition 1 the per-byte term accounts for at most $1,432/(1,432 + 3,621) = 28.3\%$ of the cost of any carriable packet. Even a cipher that cost nothing per byte, combined with a path that never copied the payload, would reduce the cost of a full-sized packet by at most 28.3% and would raise the packet rate by at most a factor of 1.40. For this configuration, it is the packet rather than the byte that determines the throughput. The crossover of `aes-256-gcm` lies even further out, at 7,906 bytes, where the bound falls to 15.3%.

ChaCha20-Poly1305 behaves differently. Its implementation written for this study has a crossover of 1,296 bytes, with an interval of $[1,165, 1,427]$, which lies

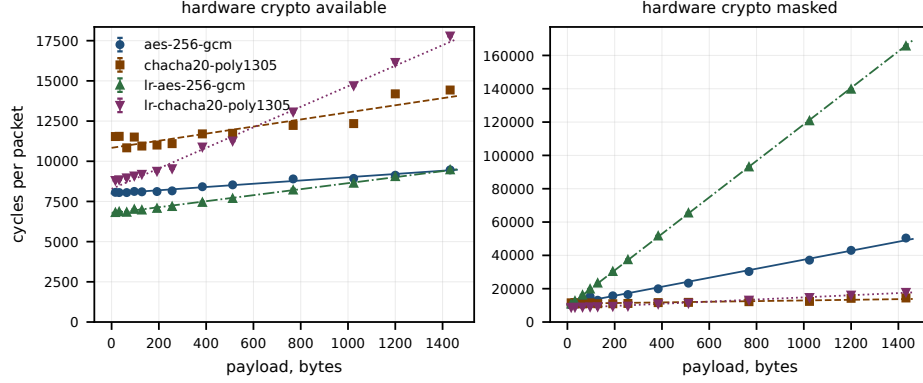


Fig. 2. Cycles per packet against payload size for the four implementations, together with the fitted cost model $C(s) = a + bs$. Error bars are bootstrap intervals on the median of 11 replicates. Left: hardware cryptographic instructions available. Right: the same instructions masked in both implementations.

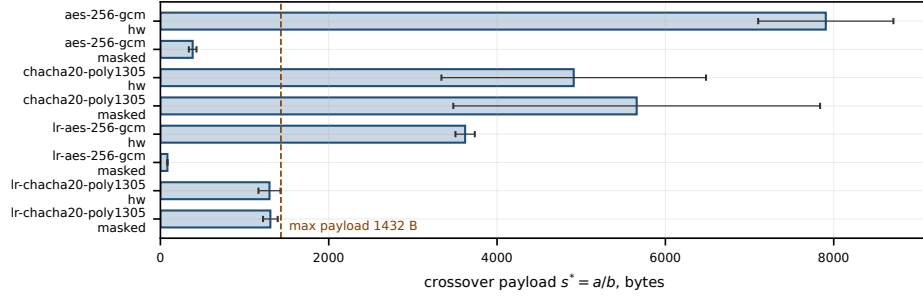


Fig. 3. Crossover payload $s^* = a/b$ of each implementation with its 95% interval. The dashed line marks $s_{\max} = 1,432$ bytes; a crossover to its right means that the fixed term dominates at every payload that the data plane can carry.

inside the carriable range. The per-byte term therefore dominates the largest packets, where it accounts for 52.5% of the cost, and above the crossover the choice of cipher implementation matters. The OpenSSL implementation has the widest intervals and the lowest coefficients of determination ($R^2 = 0.8348$ and 0.7735), so its crossover is the least precisely determined of the four.

Two Implementations of AES-256-GCM. Compared with `aes-256-gcm`, our implementation has a fixed term 0.85 times as large and a per-byte term 1.85 times as large. The difference in b has a specific cause. The OpenSSL assembly interleaves eight AES blocks by means of the vector AES extensions, whereas our implementation interleaves four with the 128-bit instructions. Our single-pass loop, which computes counter-mode encryption and GHASH in one pass over the data rather than two, recovers part of this difference but not all of

Table 1. Fitted cost model of each implementation. a is the fixed term in cycles, b the per-byte term in cycles per byte over a path that crosses the cipher twice, and $s^* = a/b$ the crossover payload in bytes with its 95% interval, propagated through the covariance of the fit. The column hw indicates whether the hardware cryptographic instructions were available.

implementation	hw	a	b	s^*	95% interval of s^*	R^2
aes-256-gcm	yes	7,995	1.011	7,906	[7,102, 8,709]	0.9791
aes-256-gcm	no	10,363	27.033	383	[338, 429]	0.9926
chacha20-poly1305	yes	10,835	2.206	4,911	[3,339, 6,483]	0.8348
chacha20-poly1305	no	10,996	1.943	5,659	[3,480, 7,838]	0.7735
lr-aes-256-gcm	yes	6,768	1.869	3,621	[3,505, 3,736]	0.9980
lr-aes-256-gcm	no	9,121	109.495	83	[82, 84]	0.9999
lr-chacha20-poly1305	yes	8,279	6.387	1,296	[1,165, 1,427]	0.9874
lr-chacha20-poly1305	no	8,382	6.416	1,307	[1,219, 1,394]	0.9953

it. Since both implementations produce identical output at every payload length, the remaining gap is a property of the instruction selection rather than of correctness. The comparison is informative in both directions: it bounds how much of the measured b is intrinsic to the algorithm rather than to one library, and it quantifies what the wider vector path contributes, which a study of a single implementation cannot reveal. The same pattern of a smaller fixed term and a larger per-byte term holds for ChaCha20-Poly1305, with ratios of 0.76 and 2.90.

Masking the Accelerator. The masked arm is not hypothetical, since it shows what the same code does on a processor without these instructions, such as an embedded or older deployment target. For `aes-256-gcm`, masking raises b from 1.011 to 27.033 cycles per byte, a factor of 26.7, and for `lr-aes-256-gcm` it raises b from 1.869 to 109.495, a factor of 58.6. This change is the entire contribution of the accelerator to the per-byte term. The fixed term rises by 2,368 and 2,353 cycles, respectively, which is the contribution of the accelerator to the per-message setup of the cipher and does not diminish with larger packets. As a result, the crossovers of both AES implementations fall deep inside the carryable range, to 383 and 83 bytes. The mask clears only the AES and carry-less multiplication instructions, which ChaCha20-Poly1305 does not use; accordingly, the two arms of both ChaCha20-Poly1305 implementations yield overlapping intervals for s^* .

4.3 The System-Call Boundary

Table 2 and Fig. 4 report the four transports at $s_{\max}$ with `aes-256-gcm`. The measured system calls per packet agree with the design of each transport: `posix` issues two, `mmsg` issues $2/B$ and `uring-sqpoll` issues none, while `uring` retains about one call per packet that batching does not remove.

Table 2. Cycles per packet and system calls per packet of the four transports at $s_{\max} = 1,432$ bytes with **aes-256-gcm**, where **sqpoll** denotes **uring-sqpoll**. **posix** issues one call per packet in each direction and has no batch depth B .

B	cycles per packet				system calls per packet			
	posix	mmsg	uring	sqpoll	posix	mmsg	uring	sqpoll
1	9,392	10,267	11,291	30,935	2.000	2.000	3.000	0.000
2	—	9,312	10,032	19,992	—	1.000	2.000	0.000
4	—	8,805	9,417	14,790	—	0.500	1.500	0.000
8	—	8,576	9,097	11,810	—	0.250	1.250	0.000
16	—	8,408	8,918	10,835	—	0.125	1.125	0.000
32	—	8,309	8,789	10,329	—	0.063	1.063	0.000
64	—	8,234	8,734	10,017	—	0.031	1.031	0.000

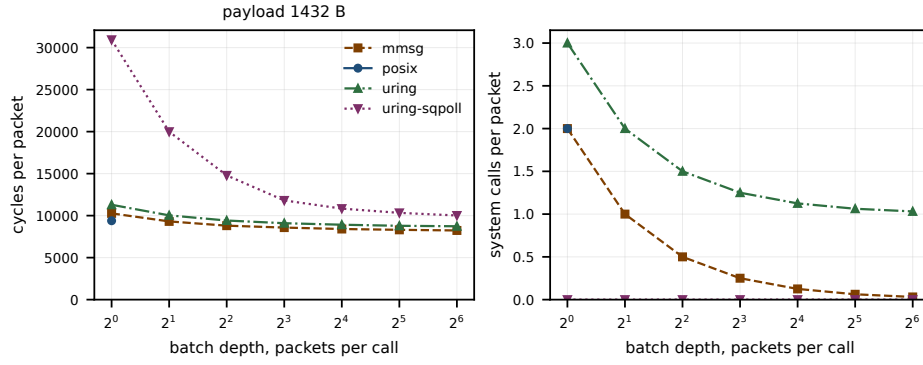


Fig. 4. Cycles per packet (left) and system calls per packet (right) against batch depth for the four transports, at $s_{\max}$ with **aes-256-gcm**. System calls are counted inside each transport rather than derived from its design, because a batch that returns short issues more calls than the design intends.

The polled path was intended to measure the contribution of the system-call boundary to a directly. At $B = 1$, however, **posix** costs 9,392 cycles per packet and **uring-sqpoll** costs 30,935 cycles, a difference of $-21,543$ cycles. The path without system calls is thus 3.3 times as expensive as the path with two calls per packet, and it approaches the batched transports only at $B = 64$, where it costs 10,017 cycles. A negative difference cannot be the cost of the boundary. The poller is an additional runnable thread, and on a machine with two physical cores, operating it costs more than the system calls it removes. On this host, the difference therefore bounds nothing, and we return to this condition in Section 5.

Batching, in contrast, behaves as expected. As the batch depth grows from 1 to 64, the cost of **mmsg** falls from 10,267 to 8,234 cycles per packet, by 19.8%, while its system calls fall from 2 to 0.031 per packet. The batched transports converge as the depth grows: once the amortised system call is small compared with the cipher, the remaining differences lie in how each transport reports

completions. The cheapest configuration, `mmsg` at $B = 64$, costs only 12.3% less than `posix`, so removing system calls reduces the cost of a full-sized packet but leaves most of it in place.

4.4 Scaling and the Memory Roof

Scaling. Each shard owns its socket, key schedule, replay window and memory arena, and shares nothing mutable with any other shard. What limits scaling is therefore not a lock but whatever n independent pipelines share: the last-level cache, the memory bandwidth and, with simultaneous multithreading, the execution resources of a physical core. We report two scaling laws, because they answer different questions. Strong scaling divides a fixed number of packets among the shards and asks whether an additional core helps with a given load, whereas weak scaling gives each shard the same number of packets and asks whether n cores absorb n times the load. Threads are created and shards are constructed and warmed up before the timed region opens, and all workers start at a barrier, so that thread creation is not charged to the data plane and no worker measures an uncontended machine because it started first.

With two shards (Fig. 5a, `aes-256-gcm` at 512 bytes), the strong speedup is 1.816, the strong and weak efficiencies are 0.908 and 0.894, and the cost per packet rises from 8,488 to 9,345 cycles. Fitting Amdahl’s law [1] to the strong speedup gives a serial fraction of 0.1012. Because the shards share nothing mutable, this fraction reflects contention for shared hardware, such as the memory system, rather than for a lock.

Memory Roof. A data plane cannot move payload faster than the memory system supplies bytes, divided by the number of times each byte is touched. In the spirit of the Roofline model [14], we measure the numerator directly with the STREAM triad [11] over a geometric range of working sets (Fig. 5b). The plateaus of the curve correspond to the levels of the cache hierarchy, and the largest working set sustains 13.7 GB/s. Each payload byte makes eight passes through memory on the forwarding path: the kernel copy on receive, the decryption, the re-encryption and the kernel copy on send each read and write it once. The resulting roof is $13.7/8 = 1.71$ GB/s of payload, a rate that this data plane cannot exceed on this machine however fast its cipher becomes. Figure 5b also indicates which level of the hierarchy serves a packet buffer of a given size.

4.5 Independent References

A cost model fitted to the measurements of a single program is consistent by construction, so we place beside it two references that this study did not write (Fig. 6). The first, `openssl speed`, times AES-256-GCM on the same machine without any packet framing and reports 0.2302 cycles per byte. The data plane pays 0.4455 cycles per byte for each pass through the same cipher, 1.94 times as much. This gap is consistent with the composition of b , which includes the

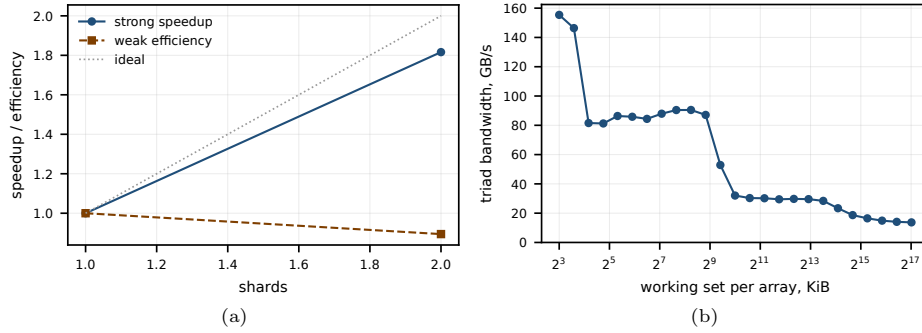


Fig. 5. (a) Strong speedup and weak efficiency against the number of shards, with `aes-256-gcm` at a payload of 512 bytes. (b) STREAM triad bandwidth against the working-set size of each array. The plateaus correspond to the levels of the cache hierarchy, and the rightmost value is the sustained bandwidth from which the memory roof is computed.

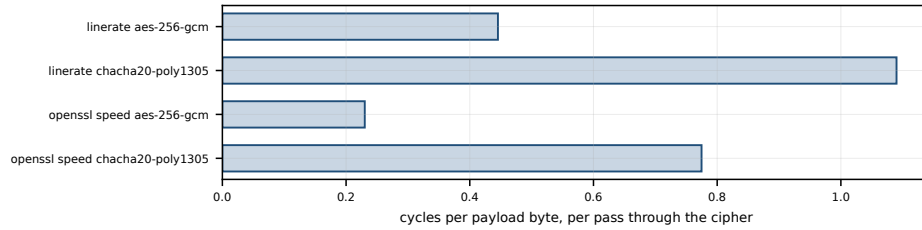


Fig. 6. Per-byte cost of the data plane (`linerate`) beside `openssl speed`, in cycles per payload byte per pass through the cipher. Since the data plane passes over each payload byte twice, its per-byte cost is halved before it is placed on this axis.

memory traffic around the cipher: when the AEAD is replaced by a null cipher, 0.2483 of its 0.9736 cycles per byte remain (Section 4.7). The second reference, `iperf3` over UDP, measures the same link without cryptography and without user-space forwarding. It reaches 6.76 Gb/s, which bounds what any encrypted data plane could achieve on this link, and the data plane reaches 3.32 Gb/s.

4.6 Isolation Tiers and Service Chaining

Of the four isolation tiers of Section 3.3, the host tier and the namespace tier could be measured on this platform (Fig. 7, left; `aes-256-gcm` at 512 bytes). The host tier costs 8,397 cycles per packet and the namespace tier 7,291 cycles, 0.868 times as much, and both issue exactly two system calls per packet. The expected ordering therefore did not appear. A namespace cannot remove work from the forwarding path, and the number of boundary crossings is identical in both tiers, so we attribute the difference to the variation between two single-configuration measurements on a shared machine rather than to isolation, and we do not derive

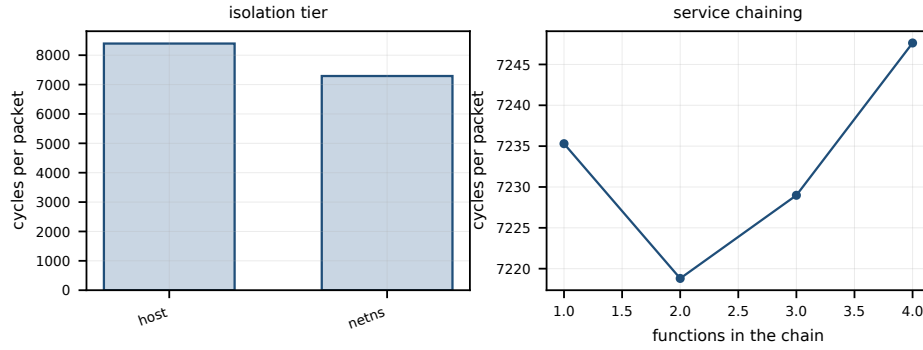


Fig. 7. Cycles per packet in the host and namespace isolation tiers (left) and along a chain of one to four encrypting network functions (right), with `aes-256-gcm` at a payload of 512 bytes.

a per-tier penalty from it. Moreover, the host tier is itself a hypervisor guest, so these tiers measure the incremental cost of each layer above the virtual machine, not the cost of virtualisation, which this setup cannot observe.

For the service function chain, built from veth-joined namespaces with one to four encrypting hops, the slope of the cost against the length of the chain is 4 cycles per hop (Fig. 7, right). This slope is the price of composition, separate from the price of the functions themselves, and at these chain lengths it lies at the noise floor of the measurement.

4.7 Validation of the Measurement

Each control carries a prediction that was fixed before the control ran, because a control whose prediction is written down afterwards cannot fail. All five controls passed, as Table 3 shows.

Null Cipher. Replacing the AEAD by a copy of the same length leaves everything else on the path unchanged: the system calls, the replay window, the header parsing and the buffers. The fixed term falls from 7,982 to 6,340 cycles, so 1,642 cycles of the fixed cost belong to the AEAD itself. This cost is per-message setup and teardown; it does not scale with the payload and cannot be amortised by sending larger packets. Without this control, a would remain a single opaque number, whereas with it, a decomposes. The per-byte term falls from 0.9736 to 0.2483 cycles per byte. Comparing the two per-byte terms is meaningful only if they can be resolved, which the control verifies rather than assumes: on this host, the per-byte cost at the largest payload exceeds a tenth of the fixed cost, so both terms are separately identifiable.

Hardware Mask. If clearing the capability bits did not raise b by a large factor, the mask would not have reached the implementation, and every masked

Table 3. Validation controls, each with the prediction that was fixed before it ran.

control	prediction	outcome	verdict
null cipher	a falls below the fixed cost with the AEAD, and b falls to roughly the cost of two memory passes	a : 7,982.0 $\rightarrow$ 6,339.7 cycles; b : 0.9736 $\rightarrow$ 0.2483 cycles per byte	pass
hardware mask	b of <code>aes-256-gcm</code> rises by at least a factor of 8	b : 0.9736 $\rightarrow$ 27.5380 (factor 28.3); a : 7,982.0 $\rightarrow$ 9,940.5	pass
idle polling	empty polls add less than 1% to the cost per packet	686.4 cycles per empty poll, fewer than 5×10^{-6} empty polls per packet	pass
forged packet	opening a forged packet costs within 25% of opening a valid one	780.4 against 777.5 cycles, ratio 1.004	pass
cipher levels	all four implementations build and forward packets	all four available	pass

measurement in Section 4.2 would be void. The mask raises b from 0.9736 to 27.538 cycles per byte, a factor of 28.3, which makes the masked arm trustworthy rather than merely present.

Instrumentation. Three further controls confirm that the harness does not measure itself. An empty poll of the socket costs 686.4 cycles, but fewer than 5×10^{-6} empty polls occur per packet, so polling adds less than 0.004 cycles to the cost of a packet. This control was originally formulated as a comparison between the cost of one empty poll and the cost of one packet, and it failed. A poll that costs a tenth of a packet but occurs once per thousand packets contaminates nothing, so we corrected the tested quantity to the amortised cost rather than relaxing the threshold. Opening a forged 512-byte packet costs 780.4 cycles, against 777.5 cycles for a valid one, a ratio of 1.004, so rejecting forged traffic is not a cheaper path through the loop. Finally, all four implementations of the design are constructible, so none of them silently drops out of the experiment.

5 Limitations

A Single Machine. All results come from one host and one microarchitecture. The code is compiled for the native architecture, so the absolute values of a and b belong to the processor on which they were measured. The crossover payload is invariant to the unit of both terms, but its transfer to other hardware follows from the structure of the model and is not demonstrated here. The host is a shared virtual machine without kernel isolation of the data-plane cores. The interleaved sweep order and the anchor check turn drift into variance and discard a sweep during which the machine changed, but they do not remove this variance.

Two Cipher Passes per Byte. The data plane decrypts and re-encrypts, which is a realistic forwarding topology and makes b large enough to be separated from a . An endpoint that performs only one of the two operations would have roughly half this b and therefore roughly twice this s^* , which would move the crossover of every accelerated AES implementation even further beyond $s_{\max}$.

The Polled Submission Path. The difference between `posix` and `uring-sqpoll` bounds the cost of the system-call boundary only when the poller does not compete with the data plane. On the measured host, the difference was negative, so the direct measurement that motivated this experiment is not available, and obtaining it requires a host on which the poller has a physical core of its own.

Isolation above a Hypervisor. Bare metal is not available on a public cloud, so the isolation tiers measure incremental costs above a virtual machine. Only two of the four tiers were measured, and they ordered themselves contrary to the expectation, so we claim no per-tier penalty.

Unmeasured Parts of the Design. The capacity of 311,881 packets per second, measured with `aes-256-gcm` at 512 bytes, implies a knee beyond which delay is dominated by queueing, but the open-loop latency sweep across this capacity returned no usable samples over the link, so the behaviour above capacity is not characterised. The cross-node part of the scaling experiment, a comparison with a kernel tunnel such as WireGuard [4], and the constants of (5) were not measured either, so Proposition 2 rests on the structure of the model rather than on measurements of this machine. A learned admission controller that predicts tail latency from the offered rate, the payload, the cipher and the network condition, and that is evaluated against a static threshold at a fixed fraction of capacity, requires the missing latency data and remains future work.

6 Conclusion

The per-packet cost of an encrypted data plane consists of a fixed term and a per-byte term, and the two call for opposite optimisations. We have therefore characterised such a data plane by the ratio of the two terms, the crossover payload $s^* = a/b$, which is invariant to the unit in which both terms are measured, and Proposition 1 turns this ratio into a bound on the benefit of any per-byte optimisation. For our AES-NI implementation of AES-256-GCM, the crossover of 3,621 bytes lies 2.5 times beyond the largest carriable payload, so the per-byte term accounts for at most 28.3% of the cost of any packet, and it is the packet rather than the byte that determines the throughput. For ChaCha20-Poly1305, in contrast, the crossover of 1,296 bytes lies inside the carriable range.

The same decomposition applies at further boundaries. At the system-call boundary, a submission path without system calls promises a direct measurement

of the cost of the boundary, but on a host with two physical cores its poller cost more than it saved. At the isolation boundary, a network namespace issued the same number of system calls as the host tier, and a chain of encrypting functions added only 4 cycles per hop. At the bus, the launch cost and the transfer term together determine a batch size below which no GPU kernel, however fast, can outperform the processor.

Two methodological choices proved essential. A second implementation of each cipher separated the cost of an algorithm from the cost of one library's version of it, and applying the mask to both implementations made the masked arm a comparison of like with like. Moreover, fixing the prediction of every control before it ran turned the controls into tests that could fail. Future work includes measuring the polled path on a host with dedicated cores, characterising the latency knee across the link, and measuring the constants of the GPU offload model.

References

1. Amdahl, G.M.: Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS Spring Joint Computer Conference. pp. 483–485 (1967)
2. Axboe, J.: Efficient IO with io_uring. https://kernel.dk/io_uring.pdf (2019)
3. Barbette, T., Soldani, C., Mathy, L.: Fast userspace packet processing. In: ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS). pp. 5–16 (2015). <https://doi.org/10.1109/ANCS.2015.7110116>
4. Donenfeld, J.A.: WireGuard: Next generation kernel network tunnel. In: Network and Distributed System Security Symposium (NDSS) (2017)
5. Dworkin, M.: Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology (2007). <https://doi.org/10.6028/NIST.SP.800-38D>
6. Gueron, S., Kounavis, M.E.: Intel carry-less multiplication instruction and its usage for computing the GCM mode. Intel White Paper (2010)
7. Hoefler, T., Belli, R.: Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In: International Conference for High Performance Computing, Networking, Storage and Analysis (SC) (2015)
8. Kent, S.: IP encapsulating security payload (ESP). RFC 4303, Internet Engineering Task Force (2005). <https://doi.org/10.17487/RFC4303>
9. Krasnov, V.: It takes two to ChaCha (Poly). Cloudflare Blog, <https://blog.cloudflare.com/it-takes-two-to-chacha-poly/> (2016)
10. Manousis, A., Sharma, R.A., Sekar, V., Sherry, J.: Contention-aware performance prediction for virtualized network functions. In: Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM). pp. 270–282 (2020). <https://doi.org/10.1145/3387514.3405868>
11. McCalpin, J.D.: Memory bandwidth and machine balance in current high performance computers. IEEE Computer Society Technical Committee on Computer Architecture Newsletter pp. 19–25 (1995)

12. Nir, Y., Langley, A.: ChaCha20 and Poly1305 for IETF protocols. RFC 8439, Internet Engineering Task Force (2018). <https://doi.org/10.17487/RFC8439>
13. Rizzo, L.: netmap: A novel framework for fast packet I/O. In: USENIX Annual Technical Conference (ATC) (2012)
14. Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM* **52**(4), 65–76 (2009). <https://doi.org/10.1145/1498765.1498785>
15. Young, E.G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H.: The true cost of containing: A gVisor case study. In: USENIX Workshop on Hot Topics in Cloud Computing (HotCloud) (2019)